

实验五 定时器 PWM 控制呼吸灯

5.1 实验目的

本实验通过 STM32F407VET6 标准外设库控制 RGB LED，实现红、绿、蓝三色依次渐亮、渐灭的呼吸灯效果。通过本实验，要求学生掌握 GPIO 复用功能、定时器 PWM 输出以及占空比调节的基本方法。

具体实验目标如下：

- (1) 掌握 RGB LED 的基本控制原理；
- (2) 理解 PWM 波形与 LED 亮度之间的关系；
- (3) 掌握 STM32F407 标准库中 GPIO 复用功能配置方法；
- (4) 掌握 TIM2 和 TIM9 输出 PWM 的基本配置流程；
- (5) 能够通过修改 PWM 比较值改变 RGB LED 的亮度；
- (6) 实现红灯、绿灯、蓝灯依次呼吸显示的实验现象。

5.2 实验简介

RGB LED 内部通常由红、绿、蓝三个独立发光单元组成。通过分别控制红、绿、蓝三个通道的亮灭，可以实现不同颜色的显示。本实验中，RGB LED 的三个颜色通道分别由 STM32 的三个 GPIO 引脚控制：红灯 -> PA0 -> TIM2_CH1，蓝灯 -> PA1 -> TIM2_CH2，绿灯 -> PE6 -> TIM9_CH2。

本实验所使用的 RGB LED 为低电平点亮，即对应 GPIO 输出低电平时 LED 点亮，输出高电平时 LED 熄灭。

如果只是使用普通 GPIO 输出高低电平，只能实现 LED 的亮和灭，无法实现亮度渐变。为了实现呼吸灯效果，本实验采用 PWM 控制方式。

PWM，即脉冲宽度调制，是一种通过改变高电平持续时间比例来调节输出平均电压的方法。PWM 的一个周期内，高电平所占比例称为占空比。占空比越大，LED 平

均获得的能量越多，看起来越亮；占空比越小，LED 越暗。

本程序中，PWM 计数周期设置为 999，因此占空比调节范围为 0 到 999。数值越大，高电平时间越长，LED 越暗。

通过让比较值从 0 逐渐增加到 999，LED 就会逐渐变暗；再让比较值从 999 逐渐减小到 0，LED 就会逐渐变亮。这种亮度由暗到亮、再由亮到暗的变化效果，就是呼吸灯效果。

5.3 硬件设计

本实验使用 STM32F407VET6 和板载 RGB LED。RGB LED 的三个颜色通道分别连接到不同的定时器 PWM 输出通道。

表 5.1 RGB LED 定时器通道

颜色	GPIO 引脚	定时器通道	点亮方式
红灯	PA0	TIM2_CH1	高电平点亮
蓝灯	PA1	TIM2_CH2	高电平点亮
绿灯	PE6	TIM9_CH2	高电平点亮

其中，PA0 和 PA1 使用 TIM2 的 PWM 输出，PE6 使用 TIM9 的 PWM 输出。由于 GPIO 引脚需要由定时器控制输出 PWM 波形，因此这三个引脚不能配置为普通输出模式，而应配置为**复用功能模式**。

5.4 实验步骤

（1）编写 RGB 部分代码

复制前面已经建立好的 STM32F407VET6 标准库工程模板，并将工程文件夹重命名为 RGB

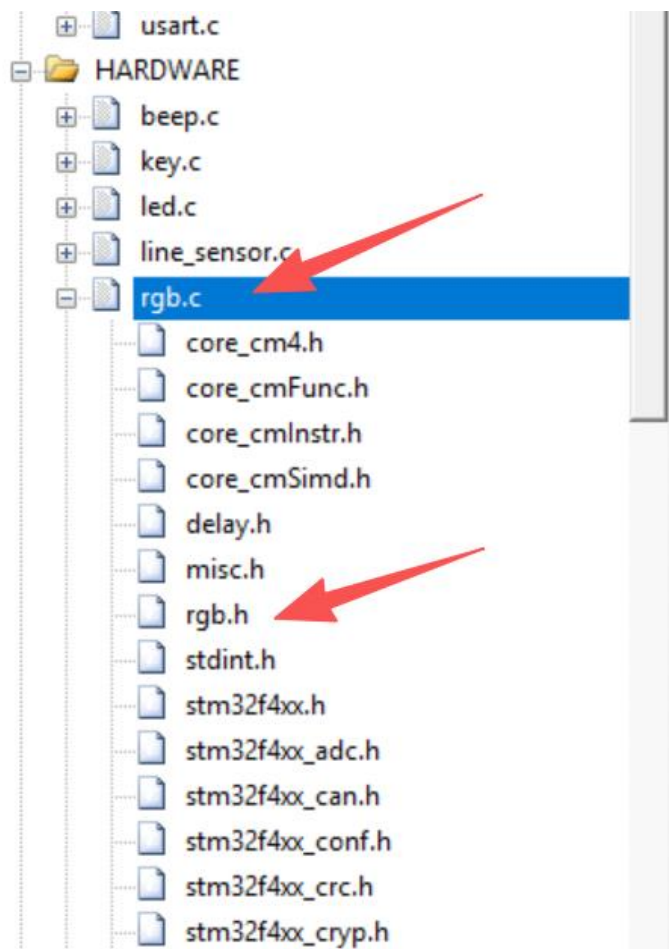


图 5.1 新建 rgb.c 和 rgb.h 文件

(2) 编写 rgb.c 和 rgb.h 代码

然后在 RGB 文件夹中新建两个文件：rgb.c , rgb.h

其中，rgb.h 用于定义 RGB LED 的引脚和函数声明；rgb.c 用于编写 GPIO 复用配置、定时器 PWM 配置和呼吸灯控制函数。

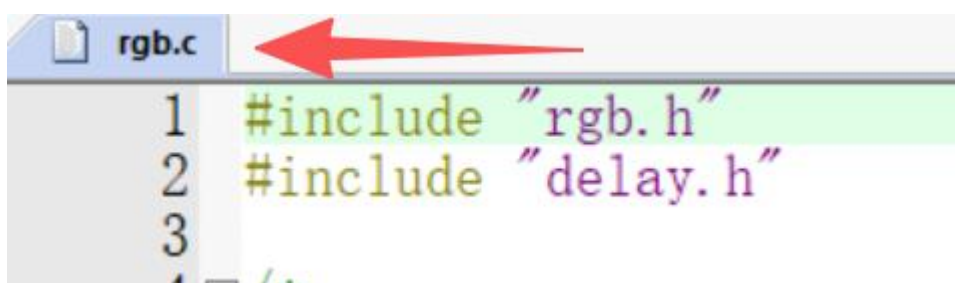


图 5.2 编写头文件

(3) 编写 rgb.h 头文件

在 `rgb.h` 文件中，首先包含 STM32 标准外设库头文件：

```
#include "stm32f4xx.h", #include <stdint.h>
```

然后定义 RGB LED 三个颜色通道的端口和引脚：

```
#define RGB_RED_PORT GPIOA, #define RGB_RED_PIN GPIO_Pin_0, #define  
RGB_BLUE_PORT GPIOA, #define RGB_BLUE_PIN GPIO_Pin_1, #define  
RGB_GREEN_PORT GPIOE, #define RGB_GREEN_PIN GPIO_Pin_6
```

由此可知，红灯连接 PA0，蓝灯连接 PA1，绿灯连接 PE6。

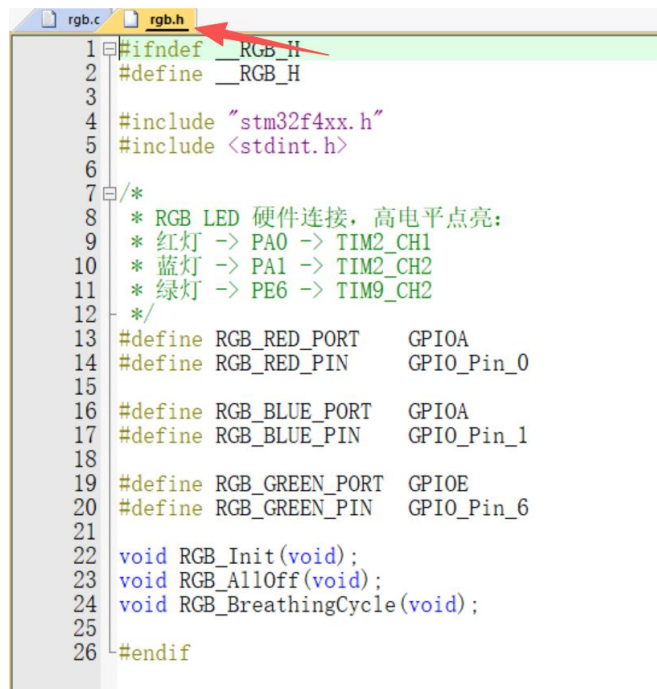
最后声明本实验需要使用的三个函数：

```
void RGB_Init(void);
```

```
void RGB_AllOff(void);
```

```
void RGB_BreathingCycle(void);
```

其中，`RGB_Init()` 用于初始化 RGB LED 所需的 GPIO 和定时器 PWM；`RGB_AllOff()` 用于关闭三个颜色通道；`RGB_BreathingCycle()` 用于执行一次红、绿、蓝依次呼吸的完整循环。

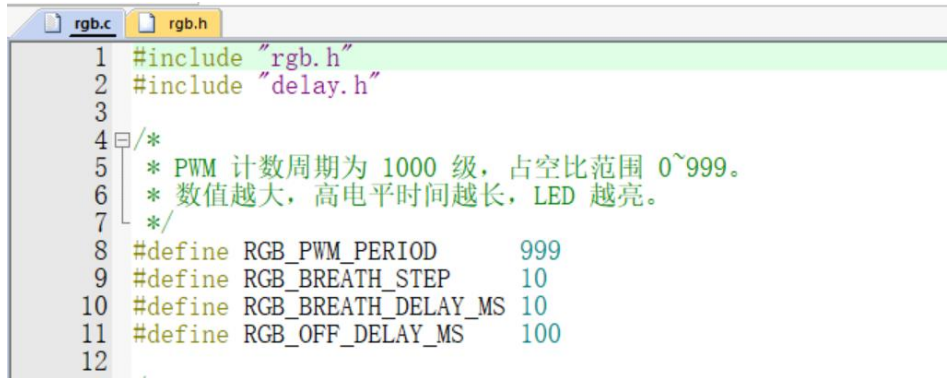


```
1 #ifndef __RGB_H__  
2 #define __RGB_H__  
3  
4 #include "stm32f4xx.h"  
5 #include <stdint.h>  
6  
7 /*  
8  * RGB LED 硬件连接，高电平点亮：  
9  * 红灯 -> PA0 -> TIM2_CH1  
10 * 蓝灯 -> PA1 -> TIM2_CH2  
11 * 绿灯 -> PE6 -> TIM9_CH2  
12 */  
13 #define RGB_RED_PORT    GPIOA  
14 #define RGB_RED_PIN     GPIO_Pin_0  
15  
16 #define RGB_BLUE_PORT   GPIOA  
17 #define RGB_BLUE_PIN    GPIO_Pin_1  
18  
19 #define RGB_GREEN_PORT   GPIOE  
20 #define RGB_GREEN_PIN    GPIO_Pin_6  
21  
22 void RGB_Init(void);  
23 void RGB_AllOff(void);  
24 void RGB_BreathingCycle(void);  
25  
26 #endif
```

图 5.3 `rgb.h` 文件

(4) 编写 rgb.c 源文件

程序开头定义了几个与呼吸灯效果有关的宏：



```
1 #include "rgb.h"
2 #include "delay.h"
3
4 /*
5  * PWM 计数周期为 1000 级，占空比范围 0~999。
6  * 数值越大，高电平时间越长，LED 越亮。
7  */
8 #define RGB_PWM_PERIOD      999
9 #define RGB_BREATH_STEP     10
10 #define RGB_BREATH_DELAY_MS 10
11 #define RGB_OFF_DELAY_MS    100
12
```

图 5.3 rgb.c 宏定义

其中，RGB_PWM_PERIOD 表示 PWM 的最大计数值为 999，因此占空比范围为 0 到 999。

RGB_BREATH_STEP 表示每次亮度变化的步长为 10。步长越小，亮度变化越细腻，但完成一次呼吸所需时间更长；步长越大，变化速度更快，但过渡可能不够平滑。

RGB_BREATH_DELAY_MS 表示每改变一次占空比后延时 10ms，用于控制亮度变化速度。

RGB_OFF_DELAY_MS 表示某一种颜色完全熄灭后，延时 100ms 再进入下一种颜色。

设置红、绿、蓝三色亮度

程序分别定义了三个静态函数，用于设置三种颜色的 PWM 比较值：

这三个函数中的 **duty** 表示 PWM 占空比对应的比较值。**duty** 越大，LED 越亮；**duty** 越小，LED 越暗。

```

/*
 * 函数作用：设置红色 LED 的 PWM 占空比。
 * 参数 duty: TIM2_CH1 的比较值，范围建议为 0~RGB_PWM_PERIOD。
 */
static void RGB_SetRed(uint16_t duty)
{
    TIM_SetCompare1(TIM2, duty);
}

/*
 * 函数作用：设置蓝色 LED 的 PWM 占空比。
 * 参数 duty: TIM2_CH2 的比较值，范围建议为 0~RGB_PWM_PERIOD。
 */
static void RGB_SetBlue(uint16_t duty)
{
    TIM_SetCompare2(TIM2, duty);
}

/*
 * 函数作用：设置绿色 LED 的 PWM 占空比。
 * 参数 duty: TIM9_CH2 的比较值，范围建议为 0~RGB_PWM_PERIOD。
 */
static void RGB_SetGreen(uint16_t duty)
{
    TIM_SetCompare2(TIM9, duty);
}

```

图 5.4 三通道占空比设置函数

关闭 RGB LED

程序使用 RGB_AllOff() 关闭三个颜色通道：

该函数将红、蓝、绿三个通道的 PWM 比较值全部设置为 0，因此三个 LED 均熄灭。

```

/*
 * 函数作用：关闭 RGB 三个颜色通道。
 * 逻辑说明：把红、绿、蓝三个 PWM 比较值都设置为 0，使输出无高电平时间。
 */
void RGB_AllOff(void)
{
    RGB_SetRed(0);
    RGB_SetBlue(0);
    RGB_SetGreen(0);
}

```

图 5.5 关闭三通道函数

GPIO 复用功能初始化

RGB LED 要输出 PWM 波形，因此 PA0、PA1、PE6 不能配置为普通 GPIO 输出，而要配置为定时器复用功能。

程序中 RGB_GPIO_Init() 函数完成 GPIO 复用配置。

首先打开 GPIOA 和 GPIOE 的时钟：

PA0 和 PA1 属于 GPIOA，PE6 属于 GPIOE，因此必须打开这两个端口的时钟。

然后配置引脚复用关系：

GPIO_PinAFConfig(GPIOA, GPIO_PinSource0, GPIO_AF_TIM2);GPIO_PinAFConfig(GPIOA, GPIO_PinSource1, GPIO_AF_TIM2);GPIO_PinAFConfig(GPIOE, GPIO_PinSource6, GPIO_AF_TIM9);这三句代码的作用是：PA0 复用为 TIM2 功能，PA1 复用为 TIM2 功能，PE6 复用为 TIM9 功能。

```
/*
 * 函数作用：初始化 RGB 灯使用的 GPIO 引脚。
 * 逻辑说明：PA0/PA1 配置为 TIM2 复用输出，PE6 配置为 TIM9 复用输出。
 */
static void RGB_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;

    /* PA0/PA1 使用 TIM2 复用功能，PE6 使用 TIM9 复用功能。 */
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA | RCC_AHB1Periph_GPIOE, ENABLE);

    GPIO_PinAFConfig(GPIOA, GPIO_PinSource0, GPIO_AF_TIM2);
    GPIO_PinAFConfig(GPIOA, GPIO_PinSource1, GPIO_AF_TIM2);
    GPIO_PinAFConfig(GPIOE, GPIO_PinSource6, GPIO_AF_TIM9);

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_DOWN;

    GPIO_InitStructure.GPIO_Pin = RGB_RED_PIN | RGB_BLUE_PIN;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Pin = RGB_GREEN_PIN;
    GPIO_Init(GPIOE, &GPIO_InitStructure);
}
```

图 5.6 RGB 初始化函数

其中，GPIO_Mode_AF 表示复用功能模式；GPIO_OType_PP 表示推挽输出；GPIO_Speed_100MHz 表示高速输出；GPIO_PuPd_DOWN 表示下拉，防止初始化时引脚状态不稳定。

TIM2 PWM 初始化

TIM2 用于控制红灯和蓝灯，其中红灯使用 TIM2_CH1，蓝灯使用 TIM2_CH2。程序中 RGB_TIM2_PWM_Init() 函数负责配置 TIM2。

首先打开 TIM2 时钟：RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2,

ENABLE); TIM2 挂载在 APB1 总线上, 因此使用 RCC_APB1PeriphClockCmd() 使能时钟。

然后配置定时器基本参数:

```
TIM_TimeBaseStructure.TIM_Period = RGB_PWM_PERIOD;
```

```
TIM_TimeBaseStructure.TIM_Prescaler = 84 - 1;
```

```
TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV1;
```

```
TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
```

```
TIM_TimeBaseInit(TIM2, &TIM_TimeBaseStructure);
```

```
static void RGB_TIM2_PWM_Init(void)
{
    TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
    TIM_OCInitTypeDef TIM_OCInitStructure;

    /* TIM2 在 APB1 上, 用来输出红灯 PA0 和蓝灯 PA1 的 PWM。 */
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);

    TIM_TimeBaseStructure.TIM_Period = RGB_PWM_PERIOD;
    TIM_TimeBaseStructure.TIM_Prescaler = 84 - 1;
    TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV1;
    TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseInit(TIM2, &TIM_TimeBaseStructure);

    TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1;
    TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
    TIM_OCInitStructure.TIM_Pulse = 0;
    TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_Low;

    TIM_OC1Init(TIM2, &TIM_OCInitStructure);
    TIM_OC1PreloadConfig(TIM2, TIM_OCPreload_Enable);

    TIM_OC2Init(TIM2, &TIM_OCInitStructure);
    TIM_OC2PreloadConfig(TIM2, TIM_OCPreload_Enable);

    TIM_ARRPreloadConfig(TIM2, ENABLE);
    TIM_Cmd(TIM2, ENABLE);
}
```

图 5.7 定时器 2 初始化

其中, TIM_Period = 999 表示计数器从 0 计数到 999, 一个周期共 1000 个计数值。

TIM_Prescaler = 84 - 1 表示对定时器时钟进行 84 分频。

TIM_CounterMode_Up 表示向上计数模式。

这里使用 PWM1 模式，初始比较值为 0，输出极性为低电平有效。因为 RGB LED 是低电平点亮，所以比较值越大，低电平时间越长，LED 越亮。

然后分别初始化 TIM2 通道 1 和通道 2：

```
static void RGB_TIM9_PWM_Init(void)
{
    TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
    TIM_OCInitTypeDef TIM_OCInitStructure;

    /* TIM9 在 APB2 上，用来输出绿灯 PE6 的 PWM。 */
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_TIM9, ENABLE);

    TIM_TimeBaseStructure.TIM_Period = RGB_PWM_PERIOD;
    TIM_TimeBaseStructure.TIM_Prescaler = 168 - 1;
    TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV1;
    TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseInit(TIM9, &TIM_TimeBaseStructure);

    TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1;
    TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
    TIM_OCInitStructure.TIM_Pulse = 0;
    TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_Low;

    TIM_OC2Init(TIM9, &TIM_OCInitStructure);
    TIM_OC2PreloadConfig(TIM9, TIM_OCPreload_Enable);

    TIM_ARRPreloadConfig(TIM9, ENABLE);
    TIM_Cmd(TIM9, ENABLE);
}
```

图 5.8 定时器 9 初始化

```
/*
 * 函数作用：完成 RGB 模块的整体初始化。
 * 逻辑说明：依次初始化 GPIO、TIM2、TIM9，最后关闭全部颜色，保证初始状态为灭。
 */
void RGB_Init(void)
{
    RGB_GPIO_Init();
    RGB_TIM2_PWM_Init();
    RGB_TIM9_PWM_Init();
    RGB_AllOff();
}
```

图 5.9 RGB 各个模块初始化

呼吸函数为 RGB_BreatheXXX()。该函数先关闭所有颜色通道，然后单独控制红灯完成一次渐亮和渐灭。

渐亮过程如下：

程序让 duty 从 0 逐渐增加到 999，每次增加 10。随着 duty 增大，PWM 占空比逐渐增大，红灯逐渐变亮。

```
/*
 * 函数作用：让红灯完成一次呼吸效果。
 * 逻辑说明：先关闭其他颜色，再逐步增大红灯占空比实现渐亮，随后逐步减小实现渐灭。
 */
static void RGB_BreatheRed(void)
{
}

/*
 * 函数作用：让绿灯完成一次呼吸效果。
 * 逻辑说明：先关闭其他颜色，再逐步增大绿灯占空比实现渐亮，随后逐步减小实现渐灭。
 */
static void RGB_BreatheGreen(void)
{
}

/*
 * 函数作用：让蓝灯完成一次呼吸效果。
 * 逻辑说明：先关闭其他颜色，再逐步增大蓝灯占空比实现渐亮，随后逐步减小实现渐灭。
 */
static void RGB_BreatheBlue(void)
{
}
```

图 5.10 三个灯光单独呼吸效果

```
/*
 * 函数作用：按红、绿、蓝顺序循环执行一次三色呼吸。
 * 逻辑说明：每个颜色完成完整的渐亮、渐灭并熄灭后，再切换到下一个颜色。
 */
void RGB_BreathingCycle(void)
{
    RGB_BreatheRed();
    RGB_BreatheGreen();
    RGB_BreatheBlue();
}
```

图 5.11 总执行函数

主函数设计

主函数代码结构较为简单：

程序首先调用 delay_init(168) 初始化系统延时函数。STM32F407 主频为 168MHz，因此这里传入 168。随后调用 RGB_Init() 初始化 RGB LED 的三个控制引脚及对应 PWM 定时器。

进入 while(1) 主循环后，程序不断调用，因此 RGB LED 会持续重复红、绿、蓝依次呼吸的效果。

5.5 实验现象

程序编译并下载到开发板后，可以观察到 RGB LED 按照以下顺序变化

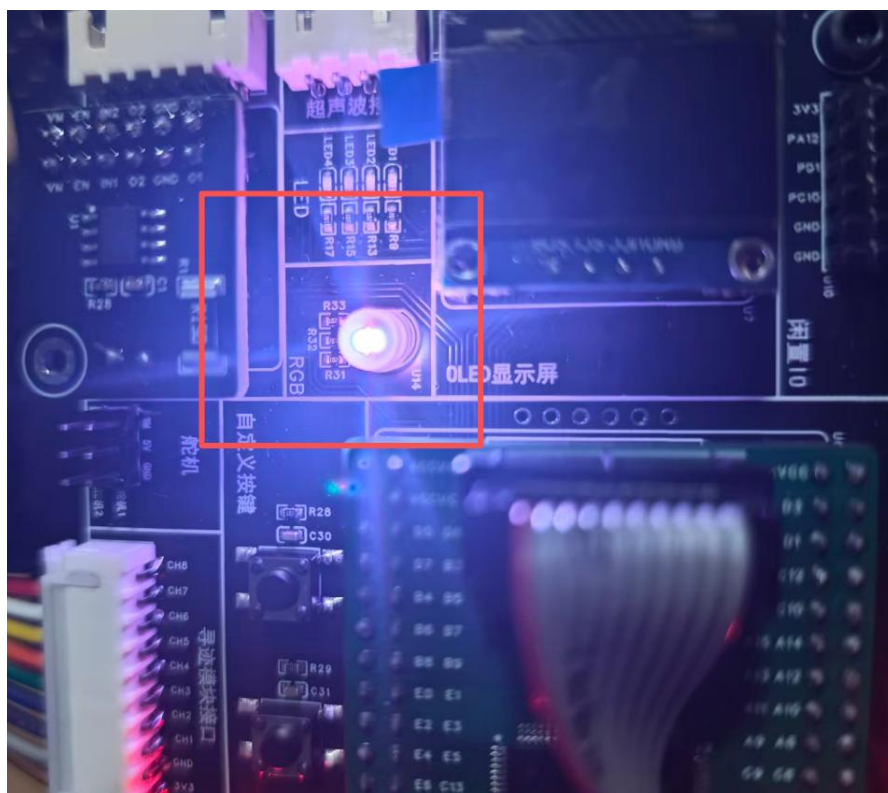


图 5.12 实物效果

实验六 定时器 PWM 控制四路电机转速

6.1 实验目的

PWM（Pulse-width modulation）是脉冲宽度调制的缩写。脉冲宽度调制是一种模拟信号电平数字编码方法。脉冲宽度调制 PWM 是通过将有效的电信号分散成离散形式从而来降低电信号所传递的平均功率的一种方式。所以根据面积等效法则，可以通过对改变脉冲的时间宽度，来等效的获得所需要合成的相应幅值和频率的波形。实现模拟电路的数字化控制可显著降低系统成本和功耗。本次实验只要目的是理解直流电机不能直接由 STM32 GPIO 引脚驱动的原因，掌握电机驱动模块的基本作用。

6.2 实验简介

6.2.1 直流电机为什么不能直接接到 STM32 GPIO

STM32 的 GPIO 引脚主要用于输出逻辑电平信号，驱动能力有限，只适合控制 LED、蜂鸣器输入端、驱动芯片控制端等小电流负载。直流电机启动瞬间电流较大，运行时还会产生反向感应电动势。如果直接把电机接到单片机 GPIO 上，不仅电机可能无法正常转动，还可能导致 GPIO 损坏。

因此，实际电机控制通常采用“单片机输出控制信号，电机驱动模块提供大电流”的结构。STM32 只负责输出 EN、DIR、PWM 等控制信号，真正给电机供电和换向的是电机驱动芯片。

6.2.2 EN、DIR、PWM 三类控制信号

本实验中，每一路电机主要涉及三类控制信号：EN、DIR 和 PWM。学生理解这三个信号后，就能看懂绝大多数直流电机驱动程序。

表 6.1 各个信号的意义

信号	中文含义	作用	可以怎样理解
EN	使能信号	打开或关闭该路电机驱动输出	相当于电机驱动的总开关
DIR	方向信号	决定电机电流方向，从而决定电机正转或反转	相当于选择电机转向
PWM	调速信号	通过快速通断改变电机平均电压，控制转速	相当于调节油门大小

6.2.3 PWM 调速原理

PWM 的全称是 Pulse Width Modulation，即脉冲宽度调制。它不是简单地让电机

一直通电或一直断电，而是在一个固定周期内快速切换高低电平。由于切换速度较快，电机不会表现为一顿一顿地转，而是表现为一个相对稳定的平均转速。

一个 PWM 周期内，高电平所占的比例称为占空比。占空比越大，电机得到的平均电压越高，通常转速越快；占空比越小，电机得到的平均电压越低，通常转速越慢。

占空比 = 高电平持续时间 / PWM 总周期 × 100%

例如，占空比为 15% 时，一个周期中只有约 15% 的时间处于驱动状态，电机低速运行；占空比为 80% 时，大部分时间处于驱动状态，电机转速明显更高。

在本程序中，EN 必须先被配置为输出并拉高，否则即使 DIR 和 PWM 已经输出，驱动模块也可能不会让电机转动。DIR 用于固定本次实验所需的转向；PWM 则由 TIM3 中断周期性刷新 GPIO 产生，用于控制电机速度。

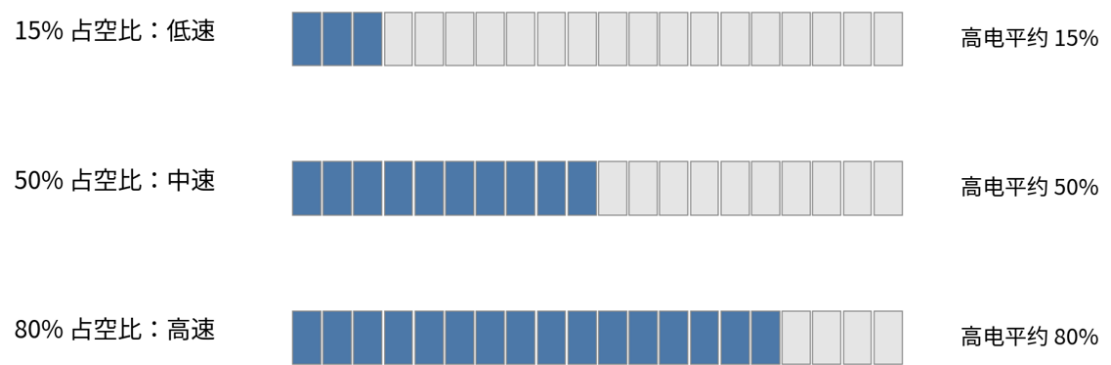


图 6.1 PWM 工作原理

6.2.3 硬件设计

图 6.2 是四个电机的整体连接图。每个电机都通过一个驱动模块控制，包含 PWM、DIRECTION 和 EN 三类信号。EN 是使能端，必须有效后电机才能工作；DIRECTION 用来控制电机转向；PWM 用来调节电机转速。每个电机还带有 ENCODER_A 和 ENCODER_B 两路编码器信号，用于后续检测电机转速和方向。

图 6.3 是单路电机驱动芯片 DRV8870 的原理图。单片机通过 IN1、IN2 给驱动芯片输入控制信号，驱动芯片再通过 OUT1、OUT2 输出较大的电流驱动电机。VM

是电机供电端，旁边的电容用于滤波稳压；ISEN 和采样电阻用于电流检测和保护。

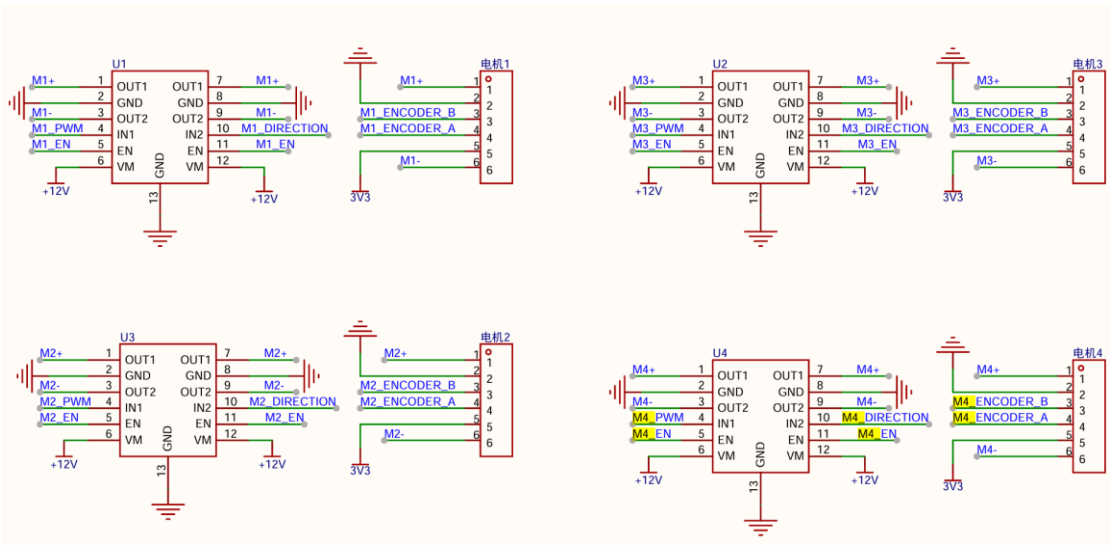


图 6.2 主板电机接口原理图

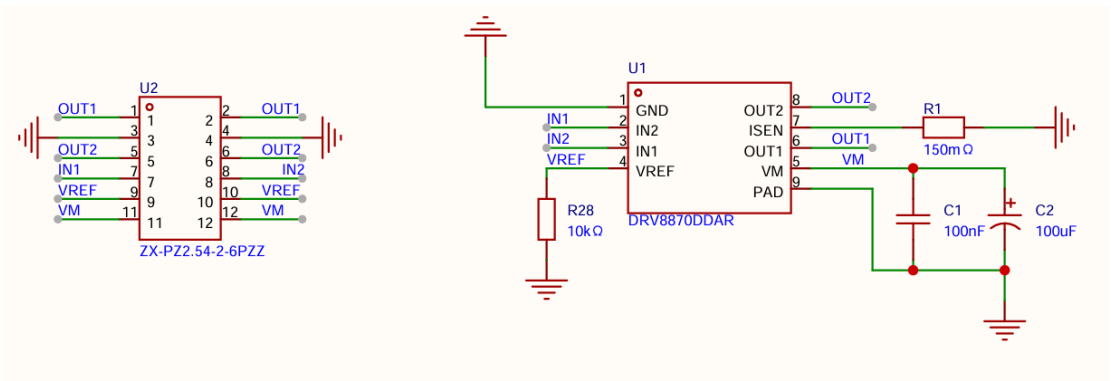


图 6.3 DRV8870DDAR 驱动原理图

6.2.4 直流电机 H 桥驱动原理

H 桥由四个开关管组成，如图 6.4 所示，电机接在中间的 OUT1 和 OUT2 之间。通过控制不同开关管导通，就可以改变电机两端电压方向，从而实现正转或反转。

左图 FORWARD 正转 时，电流大致从电源 VM 流出，经过左上方开关管，进入电机，从 OUT1 → OUT2 方向流过电机，再经过右下方开关管回到地。因此电机按正方向转动。

右图 REVERSE 反转 时，导通路径相反。电流从 VM 流出，经过右上方开关管，进入电机，从 OUT2 → OUT1 方向流过电机，再经过左下方开关管回到地。由于电机两端电压极性反过来了，所以电机反向转动。

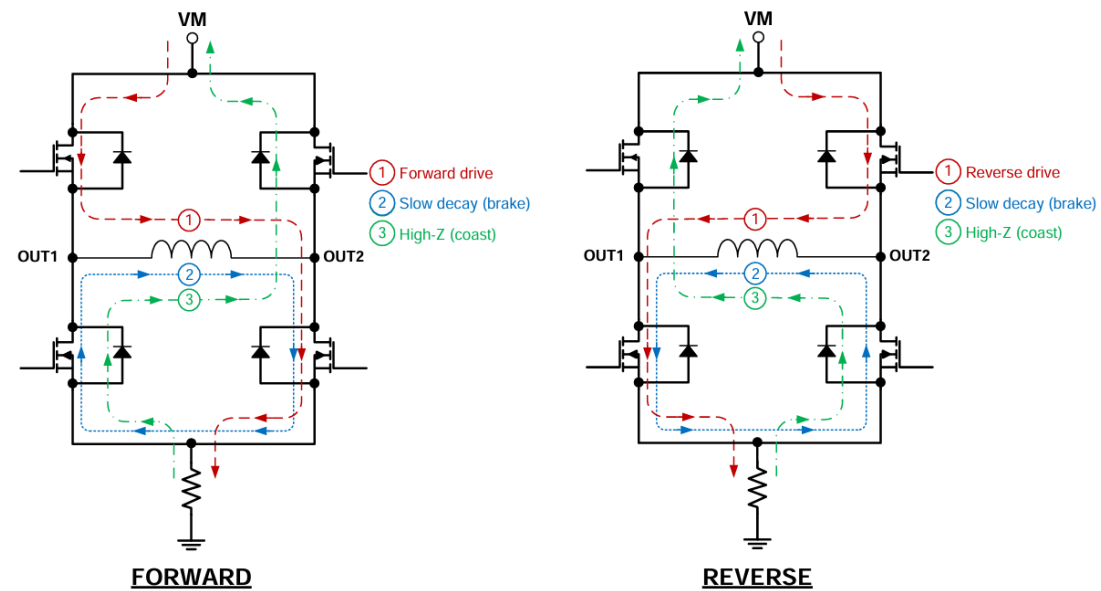


图 6.4 H 桥驱动原理

DRV8870 内部有 4 个 N 沟道 MOSFET，组成一个 H 桥。单片机只需要控制两个输入脚 **IN1** 和 **IN2**，芯片就会自动决定 **OUT1** 和 **OUT2** 输出什么电平，从而控制电机正转、反转、滑行或刹车，如表 6.2 所示。

表 6.2 H 桥控制

IN1	IN2	OUT1	OUT2	电机状态
0	0	高阻	高阻	滑行/停止驱动
0	1	低	高	反转
1	0	高	低	正转
1	1	低	低	刹车

6.3 实验步骤

本实验使用 STM32F407VET6 作为主控，通过四路电机驱动模块控制小车的四个直流电机。由于当前接线测试结果显示，四路电机的 PWM 和 DIRECTION 功能存在对调情况，因此本实验程序中将原 Mx_DIRECTION 引脚作为 PWM 控速脚，将原 Mx_PWM 引脚作为 DIR 方向脚，同时 Mx_EN 作为电机使能脚，必须先拉高后电机才能工作。

(1) 编写 MOTOR 部分代码

复制前面已经建立好的 STM32F407VET6 标准库工程模板，并将工程文件夹重命名为 MOTOR

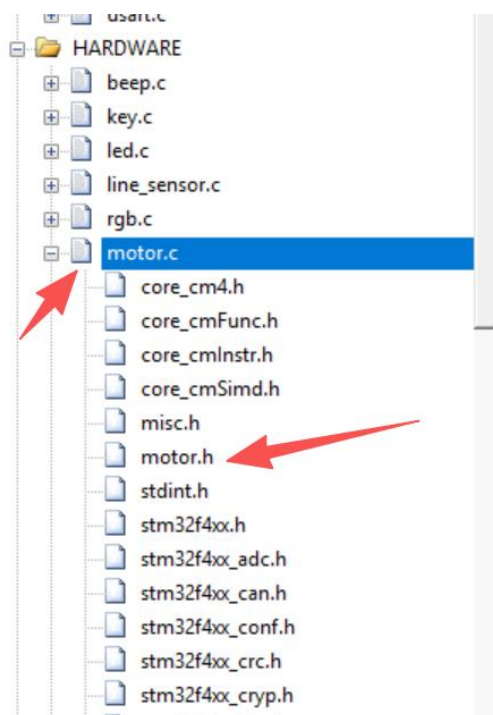


图 6.5 新建 motor.c 和 motor.h 文件

其中：

motor.h 用于定义四路电机的 PWM、DIR、EN 引脚以及函数声明；

motor.c 用于编写电机初始化、占空比设置、方向设置和 TIM3 软件 PWM 刷新程序。

(2) 编写 motor.h 引脚定义

打开 `motor.h` 文件，首先包含 STM32 标准库头文件：`#include "stm32f4xx.h"`，
`#include <stdint.h>`

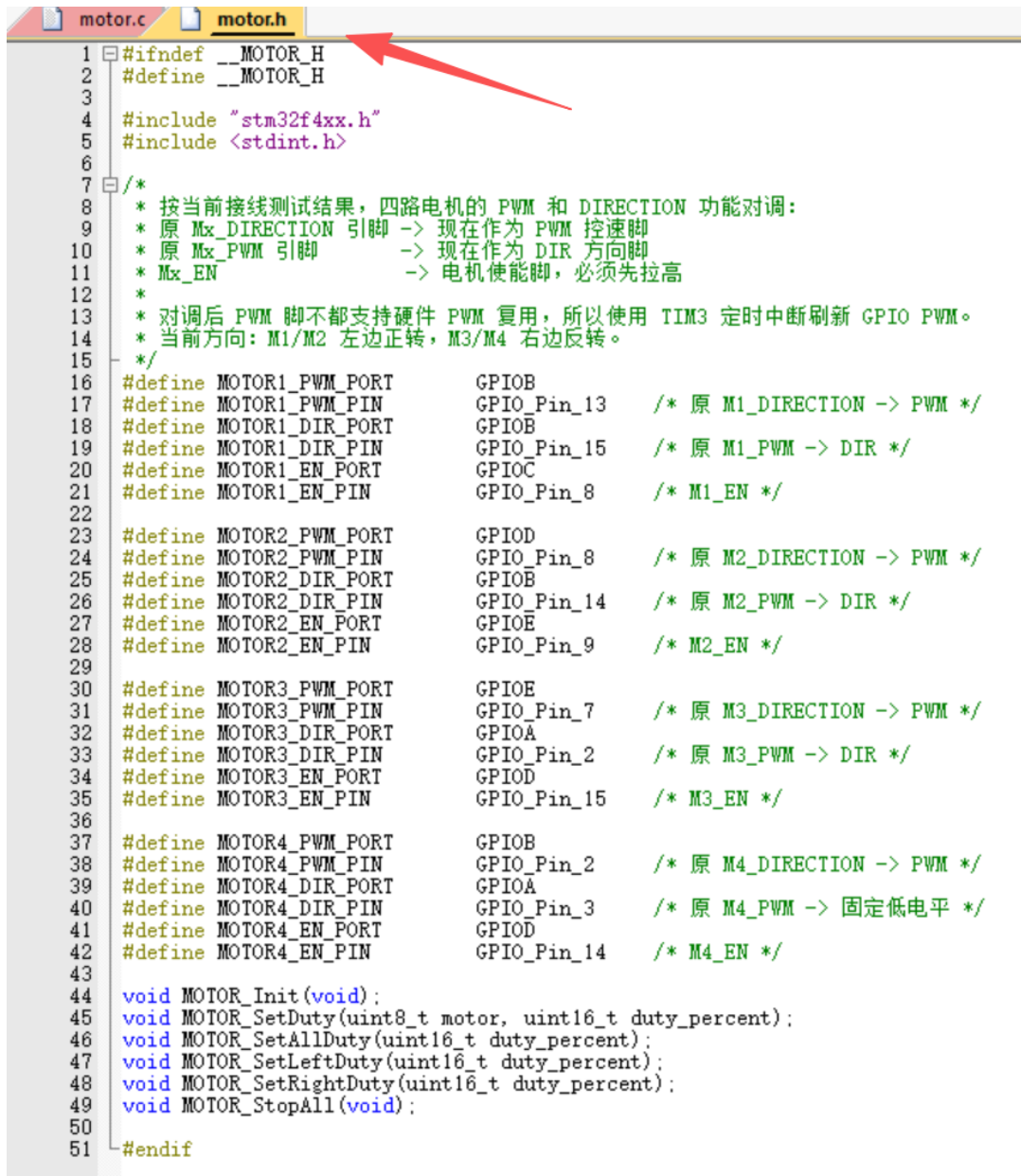
然后根据实际接线定义四路电机的 PWM、DIR 和 EN 引脚。当前代码中四路电机引脚如下：

表 6.3 电机引脚配置

电机	PWM 控速脚	DIR 方向脚	EN 使能脚
M1	PB13	PB15	PC8
M2	PD8	PB14	PE9
M3	PE7	PA2	PD15
M4	PB2	PA3	PD14

```
随后在 motor.h 中声明电机控制函数：  
  
void MOTOR_Init(void);  
  
void MOTOR_SetDuty(uint8_t motor, uint16_t duty_percent);  
  
void MOTOR_SetAllDuty(uint16_t duty_percent);  
  
void MOTOR_SetLeftDuty(uint16_t duty_percent);  
  
void MOTOR_SetRightDuty(uint16_t duty_percent);  
  
void MOTOR_StopAll(void);
```

这些函数分别用于电机初始化、单个电机占空比设置、四个电机统一调速、左侧电机调速、右侧电机调速和全部停止。



```

1  #ifndef __MOTOR_H
2  #define __MOTOR_H
3
4  #include "stm32f4xx.h"
5  #include <stdint.h>
6
7  /*
8   * 按当前接线测试结果，四路电机的 PWM 和 DIRECTION 功能对调：
9   * 原 Mx_DIRECTION 引脚 -> 现在作为 PWM 控速脚
10  * 原 Mx_PWM 引脚       -> 现在作为 DIR 方向脚
11  * Mx_EN                -> 电机使能脚，必须先拉高
12  *
13  * 对调后 PWM 脚不都支持硬件 PWM 复用，所以使用 TIM3 定时中断刷新 GPIO PWM。
14  * 当前方向：M1/M2 左边正转，M3/M4 右边反转。
15  */
16  #define MOTOR1_PWM_PORT    GPIOB
17  #define MOTOR1_PWM_PIN    GPIO_Pin_13    /* 原 M1_DIRECTION -> PWM */
18  #define MOTOR1_DIR_PORT   GPIOB
19  #define MOTOR1_DIR_PIN    GPIO_Pin_15    /* 原 M1_PWM -> DIR */
20  #define MOTOR1_EN_PORT    GPIOC
21  #define MOTOR1_EN_PIN     GPIO_Pin_8     /* M1_EN */
22
23  #define MOTOR2_PWM_PORT    GPIOD
24  #define MOTOR2_PWM_PIN    GPIO_Pin_8     /* 原 M2_DIRECTION -> PWM */
25  #define MOTOR2_DIR_PORT   GPIOB
26  #define MOTOR2_DIR_PIN    GPIO_Pin_14    /* 原 M2_PWM -> DIR */
27  #define MOTOR2_EN_PORT    GPIOE
28  #define MOTOR2_EN_PIN     GPIO_Pin_9     /* M2_EN */
29
30  #define MOTOR3_PWM_PORT    GPIOE
31  #define MOTOR3_PWM_PIN    GPIO_Pin_7     /* 原 M3_DIRECTION -> PWM */
32  #define MOTOR3_DIR_PORT   GPIOA
33  #define MOTOR3_DIR_PIN    GPIO_Pin_2     /* 原 M3_PWM -> DIR */
34  #define MOTOR3_EN_PORT    GPIOD
35  #define MOTOR3_EN_PIN     GPIO_Pin_15    /* M3_EN */
36
37  #define MOTOR4_PWM_PORT    GPIOB
38  #define MOTOR4_PWM_PIN    GPIO_Pin_2     /* 原 M4_DIRECTION -> PWM */
39  #define MOTOR4_DIR_PORT   GPIOA
40  #define MOTOR4_DIR_PIN    GPIO_Pin_3     /* 原 M4_PWM -> 固定低电平 */
41  #define MOTOR4_EN_PORT    GPIOD
42  #define MOTOR4_EN_PIN     GPIO_Pin_14    /* M4_EN */
43
44  void MOTOR_Init(void);
45  void MOTOR_SetDuty(uint8_t motor, uint16_t duty_percent);
46  void MOTOR_SetAllDuty(uint16_t duty_percent);
47  void MOTOR_SetLeftDuty(uint16_t duty_percent);
48  void MOTOR_SetRightDuty(uint16_t duty_percent);
49  void MOTOR_StopAll(void);
50
51 #endif

```

图 6.6 motor.h 文件

(3) 编写电机控制程序 motor.c

设置软件 PWM 参数

在 motor.c 开头定义软件 PWM 参数，图 6.7 这两个参数决定了软件 PWM 的精度和频率。程序将一个 PWM 周期分为 20 级，每一级占 5%。TIM3 每 50us 进入一次中断，因此 20 次中断组成一个完整 PWM 周期：

```

#define MOTOR_PWM_STEPS      20      /* 20 级占空比，每级 5% */
#define MOTOR_PWM_TICK_US    50      /* 20 * 50us = 1ms, PWM 频率约 1kHz */

static volatile uint8_t g_motor_duty_step[4] = {0, 0, 0, 0};
static volatile uint8_t g_motor_pwm_step = 0;

```

图 6.7 motor.c 宏定义

`g_motor_duty_step[4]` 用于保存四个电机的占空比等级。数组中 4 个元素分别对应 M1、M2、M3、M4。

`g_motor_pwm_step` 是当前 PWM 扫描步数，取值范围为 0 到 19。

例如，当某个电机占空比设置为 15% 时，程序会换算成 3 级左右。由于总共有 20 级，每级为 5%，因此 15% 对应 3 个高电平时间片。

程序中使用 `MOTOR_GPIO_Write()` 函数统一控制 GPIO 输出：

```

/*
 * 函数作用：封装 GPIO 输出高低电平。
 * 参数 state：非 0 输出高电平，0 输出低电平。
 */
static void MOTOR_GPIO_Write(GPIO_TypeDef *port, uint16_t pin, uint8_t state)
{
    if (state)
    {
        GPIO_SetBits(port, pin);
    }
    else
    {
        GPIO_ResetBits(port, pin);
    }
}

```

图 6.8 MOTOR_GPIO_Write() 函数

该函数的作用是：当 `state` 为 1 时输出高电平，当 `state` 为 0 时输出低电平。

这样后续程序不需要反复写 `GPIO_SetBits()` 和 `GPIO_ResetBits()`，代码更加简洁。

编写占空比换算函数

用户设置电机速度时，一般习惯使用 0% 到 100% 的占空比。但程序内部使用的是 20 级 PWM，因此需要把百分比换算成 0 到 20 的步数。

程序中还做了一个细节处理：如果输入占空比大于 0，但换算结果为 0，则强制设为 1，避免很小的占空比被直接变成停止状态。

```

/*
 * 函数作用：把百分比占空比转换成软件 PWM 的步数。
 * 逻辑说明：占空比限制在 0~100%，再映射到 0~MOTOR_PWM_STEPS；非 0 小占空比至少保留 1 步。
 */
static uint8_t MOTOR_DutyToStep(uint16_t duty_percent)
{
    uint16_t duty_step;

    if (duty_percent > 100)
    {
        duty_percent = 100;
    }

    duty_step = (duty_percent * MOTOR_PWM_STEPS + 50) / 100;
    if ((duty_percent > 0) && (duty_step == 0))
    {
        duty_step = 1;
    }

    return (uint8_t)duty_step;
}

```

图 6.9 占空比换算函数

初始化电机 GPIO，使能电机

```

/*
 * 函数作用：初始化四路电机 EN 使能引脚。
 * 逻辑说明：把 M1~M4 的 EN 脚配置为普通推挽输出，后续拉高使能驱动器。
 */
static void MOTOR_EnableGPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;

    GPIO_InitStructure.GPIO_Pin = MOTOR1_EN_PIN;
    GPIO_Init(GPIOC, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Pin = MOTOR3_EN_PIN | MOTOR4_EN_PIN;
    GPIO_Init(GPIOD, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Pin = MOTOR2_EN_PIN;
    GPIO_Init(GPIOE, &GPIO_InitStructure);
}

/*
 * 函数作用：使能全部电机驱动通道。
 * 逻辑说明：将四路 EN 引脚全部拉高，让驱动器进入可输出状态。
 */
static void MOTOR_EnableAll(void)
{
    MOTOR_GPIO_Write(MOTOR1_EN_PORT, MOTOR1_EN_PIN, 1);
    MOTOR_GPIO_Write(MOTOR2_EN_PORT, MOTOR2_EN_PIN, 1);
    MOTOR_GPIO_Write(MOTOR3_EN_PORT, MOTOR3_EN_PIN, 1);
    MOTOR_GPIO_Write(MOTOR4_EN_PORT, MOTOR4_EN_PIN, 1);
}

```

图 6.10 初始化使能引脚和使能函数

将四路 EN 全部拉高，使四路电机驱动通道处于允许工作状态。

EN 相当于电机驱动芯片的总开关。即使 PWM 和 DIR 信号正确，如果 EN 没有拉高，电机也不会转动。

初始化 PWM 和 DIR 控制引脚

接着程序初始化四路电机的 PWM 和 DIR 引脚。

这些引脚也全部配置为普通 GPIO 输出，而不是定时器复用功能输出。原因是当前 PWM 脚并不都支持硬件 PWM 复用，所以程序采用 TIM3 中断刷新 GPIO 的方式生成软件 PWM。

代码中分别初始化 GPIOA、GPIOB、GPIOD 和 GPIOE 上的控制引脚。

其中：

GPIOA 用于 M3、M4 的方向引脚；

GPIOB 用于 M1 的 PWM/DIR、M2 的 DIR、M4 的 PWM；

GPIOD 用于 M2 的 PWM；

GPIOE 用于 M3 的 PWM。

```
/*
 * 函数作用：初始化电机方向脚和软件 PWM 输出脚。
 * 逻辑说明：由于当前接线中 PWM/DIR 功能对调，所有控制脚都按普通 GPIO 输出配置。
 */
static void MOTOR_ControlGPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;

    GPIO_InitStructure.GPIO_Pin = MOTOR3_DIR_PIN | MOTOR4_DIR_PIN;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Pin = MOTOR1_PWM_PIN | MOTOR1_DIR_PIN |
                                    MOTOR2_DIR_PIN | MOTOR4_PWM_PIN;
    GPIO_Init(GPIOB, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Pin = MOTOR2_PWM_PIN;
    GPIO_Init(GPIOD, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Pin = MOTOR3_PWM_PIN;
    GPIO_Init(GPIOE, &GPIO_InitStructure);
}
```

图 6.11 初始化电机引脚函数

```

/*
 * 函数作用：设置四路电机的固定运动方向。
 * 逻辑说明：M1/M2 作为左侧正转，M3/M4 作为右侧反转，形成当前小车前进方向配置。
 */
static void MOTOR_SetDirection(void)
{
    /* 左边 M1/M2 正转：DIR=0。 */
    MOTOR_GPIO_Write(MOTOR1_DIR_PORT, MOTOR1_DIR_PIN, 0);
    MOTOR_GPIO_Write(MOTOR2_DIR_PORT, MOTOR2_DIR_PIN, 0);

    /* M3 反转：DIR=1, PWM 低电平期间驱动。M4 反转：PA3=0, PB2 输出 PWM。 */
    MOTOR_GPIO_Write(MOTOR3_DIR_PORT, MOTOR3_DIR_PIN, 1);
    MOTOR_GPIO_Write(MOTOR4_DIR_PORT, MOTOR4_DIR_PIN, 0);
}

/*
 * 函数作用：一次性刷新四路电机的软件 PWM 输出电平。
 * 参数 m1 m4：分别表示四路 PWM GPIO 当前时刻应输出的高低电平。
 */
static void MOTOR_SetPWMPins(uint8_t m1, uint8_t m2, uint8_t m3, uint8_t m4)
{
    MOTOR_GPIO_Write(MOTOR1_PWM_PORT, MOTOR1_PWM_PIN, m1);
    MOTOR_GPIO_Write(MOTOR2_PWM_PORT, MOTOR2_PWM_PIN, m2);
    MOTOR_GPIO_Write(MOTOR3_PWM_PORT, MOTOR3_PWM_PIN, m3);
    MOTOR_GPIO_Write(MOTOR4_PWM_PORT, MOTOR4_PWM_PIN, m4);
}

```

图 6.12 设置电机默认方向

根据代码注释，M1 和 M2 为左侧电机，设置为正转；M3 和 M4 为右侧电机，设置为反转。这样做的原因是小车左右两侧电机安装方向通常相反。如果左右两边都给相同方向信号，小车可能一边向前、一边向后，导致原地打转。

初始化 TIM3 定时中断

本实验使用 TIM3 定时器产生周期性中断。每次进入 TIM3 中断，程序就刷新一次四个 PWM 输出脚的电平。

```

/*
 * 函数作用：初始化 TIM3 定时中断，用于产生软件 PWM 的时间基准。
 * 逻辑说明：TIM3 每 50us 进入一次中断，20 个中断周期组成约 1kHz 的软件 PWM 周期。
 */
static void MOTOR_TIM3_Init(void)
{
    TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
    NVIC_InitTypeDef NVIC_InitStructure;

    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3, ENABLE);

    TIM_TimeBaseStructure.TIM_Period = MOTOR_PWM_TICK_US - 1;
    TIM_TimeBaseStructure.TIM_Prescaler = 84 - 1;
    TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV1;
    TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseStructure.TIM_RepetitionCounter = 0;
    TIM_TimeBaseInit(TIM3, &TIM_TimeBaseStructure);

    TIM_ClearITPendingBit(TIM3, TIM_IT_Update);
    TIM_ITConfig(TIM3, TIM_IT_Update, ENABLE);

    NVIC_InitStructure.NVIC_IRQChannel = TIM3_IRQn;
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 2;
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    NVIC_Init(&NVIC_InitStructure);

    TIM_Cmd(TIM3, ENABLE);
}

```

图 6.13 配置定时器 3

```

/*
 * 函数作用：完成电机模块的整体初始化。
 * 逻辑说明：开启 GPIO 时钟，初始化并拉高 EN，配置控制脚，设置方向和停止状态，最后启动 TIM3 软件 PWM。
 */
void MOTOR_Init(void)
{
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA |
                           RCC_AHB1Periph_GPIOB |
                           RCC_AHB1Periph_GPIOC |
                           RCC_AHB1Periph_GPIOD |
                           RCC_AHB1Periph_GPIOE, ENABLE);

    /* EN 必须最先配置并拉高。 */
    MOTOR_EnableGPIO_Init();
    MOTOR_EnableAll();

    MOTOR_ControlGPIO_Init();
    MOTOR_SetDirection();
    MOTOR_StopAll();

    MOTOR_TIM3_Init();
}

```

图 6.14 各个模块初始化

```

/*
 * 函数作用：设置指定电机的目标占空比。
 * 参数 motor：电机编号 1~4；duty_percent：目标占空比百分比，超过 100 会被限制为 100。
 */
void MOTOR_SetDuty(uint8_t motor, uint16_t duty_percent)
{
    uint8_t duty_step;

    duty_step = MOTOR_DutyToStep(duty_percent);

    switch (motor)
    {
        case 1:
            ??
            break;

        case 2:
            ??
            break;

        case 3:
            ??
            break;

        case 4:
            ??
            break;

        default:
            break;
    }
}

```

图 6.15 设置电机占空比

其中，motor 表示电机编号，可以是 1、2、3、4；duty_percent 表示占空比百分比，范围为 0 到 100。（需要自己填写）

```

/*
 * 函数作用：同时设置四路电机的目标占空比。
 * 逻辑说明：内部依次调用 MOTOR_SetDuty 更新 M1~M4 的软件 PWM 步数。
 */
void MOTOR_SetAllDuty(uint16_t duty_percent)
{
    MOTOR_SetDuty(1, duty_percent);
    MOTOR_SetDuty(2, duty_percent);
    MOTOR_SetDuty(3, duty_percent);
    MOTOR_SetDuty(4, duty_percent);
}

/*
 * 函数作用：设置左侧两路电机的目标占空比。
 * 逻辑说明：当前 M1/M2 归为左侧电机，用于统一控制左侧速度。
 */
void MOTOR_SetLeftDuty(uint16_t duty_percent)
{
    MOTOR_SetDuty(1, duty_percent);
    MOTOR_SetDuty(2, duty_percent);
}

/*
 * 函数作用：设置右侧两路电机的目标占空比。
 * 逻辑说明：当前 M3/M4 归为右侧电机，用于统一控制右侧速度。
 */
void MOTOR_SetRightDuty(uint16_t duty_percent)
{
    MOTOR_SetDuty(3, duty_percent);
    MOTOR_SetDuty(4, duty_percent);
}

/*
 * 函数作用：停止全部电机输出。
 * 逻辑说明：占空比清零后重新保持 EN 和方向状态，并把各 PWM 脚置到不会误驱动的空闲电平。
 */
void MOTOR_StopAll(void)
{
    MOTOR_SetAllDuty(0);
    MOTOR_EnableAll();
    MOTOR_SetDirection();

    /* 左边停在低电平；M3 停在高电平；M4 停在低电平，避免误驱动。 */
    MOTOR_SetPWMPins(0, 0, 1, 0);
}

```

图 6.16 一些辅助函数

编写 TIM3 中断服务函数

本实验最关键的部分是 TIM3_IRQHandler()。

TIM3 每 50us 进入一次中断。每次中断中，程序会判断当前 PWM 步数是否小于设定占空比步数。

对于 M1 和 M2：

```
m1_pwm = (g_motor_pwm_step < g_motor_duty_step[0]) ? 1 : 0;
```

```
m2_pwm = (g_motor_pwm_step < g_motor_duty_step[1]) ? 1 : 0;
```

这表示当当前 PWM 步数小于设定步数时，PWM 输出高电平；否则输出低电平。

例如，占空比为 15%，换算后为 3 步。则在 20 个时间片中，前 3 个时间片输出高电平，后 17 个时间片输出低电平，形成约 15% 占空比的 PWM。对于 M3，程序逻辑相反。

```
/*
 * 函数作用：TIM3 中断服务函数，周期性刷新四路软件 PWM。
 * 逻辑说明：根据当前 PWM 步号和各电机目标步数计算输出电平，刷新 GPIO 后推进步号，满 20 步回到 0。
 */
void TIM3_IRQHandler(void)
{
    uint8_t m1_pwm;
    uint8_t m2_pwm;
    uint8_t m3_pwm;
    uint8_t m4_pwm;

    if (TIM_GetITStatus(TIM3, TIM_IT_Update) != RESET)
    {
        TIM_ClearITPendingBit(TIM3, TIM_IT_Update);

        /* 左边正转：PWM 高电平期间驱动。 */
        m1_pwm = (g_motor_pwm_step < g_motor_duty_step[0]) ? 1 : 0;
        m2_pwm = (g_motor_pwm_step < g_motor_duty_step[1]) ? 1 : 0;

        /* M3 低电平期间驱动，M4 高电平期间驱动。 */
        m3_pwm = (g_motor_pwm_step < g_motor_duty_step[2]) ? 0 : 1;
        m4_pwm = (g_motor_pwm_step < g_motor_duty_step[3]) ? 1 : 0;

        MOTOR_SetPWMPins(m1_pwm, m2_pwm, m3_pwm, m4_pwm);

        g_motor_pwm_step++;
        if (g_motor_pwm_step >= MOTOR_PWM_STEPS)
        {
            g_motor_pwm_step = 0;
        }
    }
}
```

图 6.17 定时器中断

(4) 编写 main.c 测试程序

打开 main.c，包含实验需要的头文件

6.4 下载程序并进行低速测试

程序编译无误后，将 ST-Link 与开发板连接，点击 Download 下载程序。

第一次测试时，必须让小车轮子离地，或将小车放在支架上，避免突然运动。程序运行后，观察四个电机是否低速转动。

方向测试

观察小车四个轮子的转向。

正常情况下，左侧 M1、M2 与右侧 M3、M4 虽然程序设置方向不同，但小车整体应表现为同一个前进方向。原因是左右两侧电机安装方向相反，所以程序中 M1/M2 设置为正转，M3/M4 设置为反转。

如果出现以下情况，应调整方向：

- （1）小车原地打转：说明左右两侧方向不一致。
- （2）某一个轮子反转：说明该电机 DIR 电平或电机线接反。
- （3）小车后退：说明整体方向与预期相反，可以统一调整方向逻辑。

调整方向时，可以修改 `MOTOR_SetDirection()` 中对应电机 DIR 引脚的输出电平。